WEEK 5 · DAY 3 · RULE BOOK

Regression Forecasts + Honest Ranges

Follow it step by step. Every common error and its fix is here.

Your goal today: build the code book's regression forecast **without leaking the future**, score it **like with like**, fence it with a 90% band you **check**, and fill in your Checkpoint 4 table. **14 green ✅ ticks** to collect. Time: about 50 minutes, then Checkpoint 4. Keep this page open next to Colab.

Before you start (2 minutes)

- ☐ Open `W5D3-exercise` in Colab → **File** → **Save a copy in Drive**.
- ☐ Run **STEP 0**. Wait for ✅ `Ready!`
- ☐ Run the data cell and the 🧰 **toolbox** cell. Everything after them needs them.
- ☐ Run every ▶️ cell in order. Each ✍️ cell depends on the cells above it.

The leak rule (ask it of every feature)

"Could I compute this on Monday morning, before this week's tickets arrive?" If not, it's a leak. Delete it.

✅ Known before the week starts	❌ Leaks the future
`df["tickets_raised"].shift(1)` : last week	`df["tickets_raised"].shift(-1)` : next week (tomorrow's newspaper)
`df["tickets_raised"].shift(52)` : same week last year	`rolling(4, center=True)` : mixes in future weeks
`active_vsat_terminals` : from the rollout plan	this week's own ticket count as a feature
`sin52` , `cos52` : the calendar is known years ahead	anything fitted on the test weeks

The exercise, step by step

Task	What you do	The tool you need
A1	Last week's count on this week's row	`LAG = 1` (a positive number looks back)
A2	Rows lost by the feature table	`len(tickets) - len(feat)`
A3	Hold out the last 8 weeks	`H = 8`

Task	What you do	The tool you need
B1	Fit and score the regression	`TARGET = "target"`
B2	Read the terminals coefficient	`coefs["terminals"]`
B3	Naive with the same hint	`NAIVE_COL = "tickets_lag1"`
B4	Did the regression beat the best baseline?	`True` or `False` (no quotes)
C1	The code book's band and its coverage	`Z = 1.645`
C2	A band from honest past misses	`Q = 0.9` (a fraction, not 90)
C3	The trap: summer misses only	`SUMMER_START = "2025-05-19"`
C4	The rate forecast's honest band	`ORIGIN = len(tickets) - H`
C5	Is raising z until it fits honest?	`True` or `False`
D1	Your recommendation	the method's name, in quotes
🌶️ **Bonus**	Amaka's sentence for 3 Nov	a pair `(low, high)`

Today's tools (from the 🧰 toolbox cell)

```
feat = make_features(tickets)                          # lag1, lag52, terminals, sin52,
cos52, target
reg = LinearRegression().fit(train_f[X_COLS], train_f["target"])
pred = reg.predict(test_f[X_COLS])                     # one-week-ahead forecasts
mae(pred, test_f["target"])                            # on a typical week, how many
tickets off?
walk_forward_errors(train_f, X_COLS, first=12)         # honest past misses of the
regression
rate_past_errors(tickets, ORIGIN, h=8)                 # honest past misses of the 8-week
rate forecast
coverage(forecast, half, actual)                       # how many weeks fell inside
forecast ± half
```

Like with like: three questions before you compare two scores

1. **Same weeks?** Both scored on 3 Nov – 22 Dec 2025.
2. **Same horizon?** "1 week ahead" (it saw last week's real count) is easier than "8 weeks, blind".
3. **Same information?** Write down what each method knew. Then beat the **best** baseline, not the weakest.

The honest band recipe

1. Walk forward through the past: fit on the weeks **before** each week, forecast it, write down the miss.

2. Make sure those weeks include the **hard** weeks you'll forecast (season changes!).
3. Half-width = the **90th percentile** of the absolute misses: `np.quantile(np.abs(misses), 0.9)`.
4. Band = forecast ± half-width. **Check coverage** on weeks it never saw: about 7 or 8 of 8 for a 90% band.
5. Never pick the width by looking at the test weeks.

Say it like Amaka

> *"Week of 3 Nov: most likely around **56**, plausibly **49 to 63**; expect bigger misses at season changes."*

Most likely · a range · what could blow it up. Never "90% sure" about a band nobody checked.

When things go wrong: error → fix

You see	It means	Fix
`AssertionError: ✍️ Fill in … first`	A `___` is still there	Replace it, run the cell again
`AssertionError: ✍️ Finish A3 first` (or B1, C2…)	An earlier task is still blank	Go back and finish that task
`NameError: name 'make_features' …` / `'mae'` / `'coverage'`	You skipped the toolbox	Run the 🧰 toolbox cell
`NameError: name 'target' …`	Column names need quotes	`TARGET = "target"`
`KeyError: 'tickets_raised'` in B1	The feature table calls it `target`	`TARGET = "target"`
`ValueError: Quantiles must be in the range [0, 1]`	You wrote `90`	Write `0.9`
`ValueError: Invalid frequency: H`	Old code with capital `"H"`	Use lowercase `"h"`; this weekly file uses `"W-MON"`
A1 says "tomorrow's newspaper"	You typed `-1`	A positive number looks back: `1`
Everything is weird	–	**Runtime → Restart session and run all**

Rules of thumb you'll keep forever

1. **Every feature known before the forecast is made.** `shift(+n)` only.
2. **Read the coefficient signs.** A sign that makes no physical sense is a bug until proven otherwise.
3. **Compare like with like:** same weeks, same horizon, same information. Beat the **best** baseline.
4. **A band is a promise.** Build it from past misses that include the hard weeks, then check its coverage.
5. **Never tune on the test.** Not z, not the window, not anything.

6. **Say it with a range** and name the thing that could blow it up.

🏁 **Finished?** Run `scoreboard()`. Try the 🌶️ sentence, then get your Checkpoint 4 table ready for the trainer.